SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 0614 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15 **Total Marks:25**

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Variganji Sumanth Babu / 192524447, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

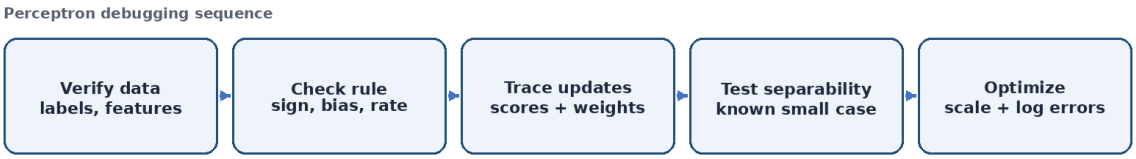
Signature of the student:

V. Sumanth Babu

Annexure A - Short Answer Test

QUESTION 1

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)



Why a separable dataset should converge

For a finite, strictly linearly separable dataset, the standard perceptron convergence theorem guarantees convergence when the update rule is implemented correctly. Therefore, repeated failure usually means that an assumption is violated or the implementation differs from the standard algorithm, rather than weight initialization alone being responsible.

Debugging checklist

1. Verify labels and scores: use labels $y \in \{-1,+1\}$ for the standard rule and compute score = $w^Tx + b$. Check that the predicted class uses the correct sign and that label mapping is consistent throughout training and evaluation.

2. Check the update rule: when $y(w^Tx+b) \leq 0$, update $w \leftarrow w + \eta yx$ and $b \leftarrow b + \eta y$, with $\eta > 0$. A missing sign, omitted bias update, incorrect condition, stale score, or updating only some weights can prevent convergence.

3. Inspect data and preprocessing: confirm there are no mislabeled examples, NaNs, inconsistent scaling, unintended train/test leakage, or a transformation that makes the processed samples non-separable. Test on a tiny synthetic dataset with a known separating line.

4. Log each epoch: record number of mistakes, weight norm, bias, and minimum signed margin. Check that an actual mistake triggers an update. Confirm the loop termination condition and learning rate are not accidentally zero or overwritten.

Optimization and interpretation

Feature standardization can improve numerical conditioning and make learning-rate choices easier, though it does not repair a faulty rule. A positive fixed learning rate changes update scale but should not by itself destroy the convergence guarantee of the standard perceptron on separable data. If the real data are noisy or not truly separable, use a pocket/averaged perceptron or a regularized linear classifier and select it with validation. This distinction prevents endless iteration on a dataset that violates the theorem's assumptions.

Conclusion: first verify the separability assumption and update mathematics; tune the learning rate only after the implementation and labels are correct.

A useful unit test is to train on a tiny two-class dataset whose points can clearly be separated by a line. Manually compute the score and margin for the first few samples and confirm that each misclassified sample triggers an update. If bias is represented as an extra weight, append a constant feature x0=1 and update that weight consistently. The perceptron can still cycle if the processed labels contain contradictory cases or if the loop mistakenly evaluates the old score after updating the parameters; epoch logs make these defects visible.

QUESTION 2

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)



Diagnose before changing the optimizer

1. Establish a baseline: plot training and validation loss, verify the data split and labels, and confirm that the loss decreases on a tiny batch that the model should be able to overfit. If even this fails, investigate the forward pass, loss function, gradient calculation, and masking/broadcasting errors.

2. Check gradients and activations: log gradient norms layer by layer. Very small gradients in early layers suggest vanishing gradients; exploding norms suggest instability. Sigmoid/tanh may saturate in deep hidden layers. ReLU or GELU, suitable initialization, and residual connections can help.

3. Inspect initialization and scale: use Xavier/Glorot initialization for sigmoid/tanh layers or He initialization for ReLU-like layers. Standardize numeric inputs and consider batch normalization or layer normalization. Ensure that target scaling and loss reduction are correct.

Optimization techniques

Learning rate: run a small learning-rate sweep. A rate that is too low makes training crawl; one that is too high causes oscillation or divergence. Use a schedule such as cosine decay or reduce-on-plateau after selecting a stable starting value.

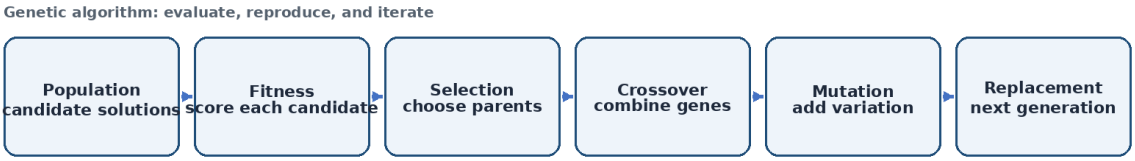
Optimizer and stability: compare SGD with momentum and Adam/AdamW. Clip gradients if norms spike, use appropriate batch sizes, and monitor validation rather than training loss alone. Early stopping prevents wasted epochs after validation stops improving.

A plateau does not prove that training is trapped in a local minimum; saddle points, flat regions, noisy gradients, or data/implementation errors can look similar. Diagnose the symptom using loss curves and gradient logs before selecting a remedy. Choose the final model using validation performance and confirm it once on a held-out test set.

A finite-difference gradient check can compare a numerical estimate of a parameter derivative with the backpropagated gradient on a very small model; a large mismatch points to an implementation bug. Also inspect label balance, incorrect target encoding, batch shuffling, and whether the loss is averaged or summed as intended. Change one setting at a time and keep a record of the experiment, because changing activation, learning rate, normalization, and optimizer simultaneously makes the true cause difficult to identify.

QUESTION 3

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)



Systematic debugging

1. Validate chromosome encoding: ensure every chromosome decodes into a legal hypothesis and that crossover/mutation preserve constraints or repair invalid offspring. Test the encoding with known candidates before running a full population.

2. Verify the fitness function: hand-calculate fitness for several chromosomes. Confirm that maximizing versus minimizing is consistent, penalties have the intended sign, constraints are handled, and the score is deterministic for a fixed candidate. A coding error or noisy score can reward the wrong solution.

3. Reproduce the inconsistency: run fixed random seeds and log best/mean fitness, population diversity, mutation count, and the best chromosome each generation. Compare multiple seeds and check stopping criteria. Low diversity with stalled fitness indicates likely premature convergence.

Optimization strategies

Population diversity: increase population size or use diversity-aware selection; tune mutation rate and consider adaptive mutation that increases when fitness stagnates. Too little mutation loses exploration, while too much destroys useful combinations.

Selection pressure and crossover: use tournament or rank selection to avoid one early high-fitness individual dominating the whole population. Tune crossover to the encoding and test that offspring inherit meaningful substructures.

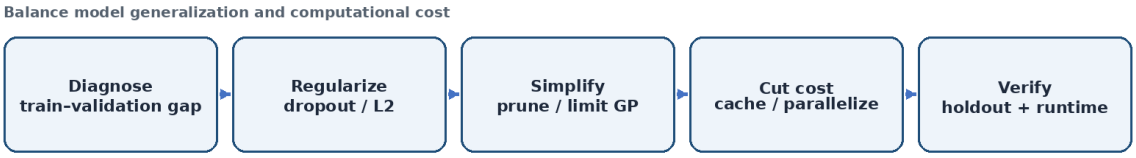
Elitism and reproducibility: copy a small number of top candidates unchanged to prevent loss of the best score, but avoid excessive elitism that removes diversity. Use seeded runs, document hyperparameters, and report distributions across independent runs rather than a single lucky result.

Success criteria: the best fitness should be non-decreasing when elitism is used, diversity should not collapse immediately, and independent runs should meet an acceptable quality range. If not, inspect fitness and encoding before increasing compute.

A practical experiment compares several configurations under the same evaluation budget: baseline parameters, higher mutation, lower selection pressure, and elitism. Record the best fitness and the median/variation across runs, not just the single best run. If all candidates have nearly identical chromosomes but fitness remains mediocre, increase exploration or restart part of the population. If fitness fluctuates wildly, check stochastic fitness evaluation, constraint handling, and whether candidate scores are comparable across generations.

QUESTION 4

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)



Debugging the accuracy and cost problem

1. Measure the generalization gap: compare training, validation, and independent test performance. High training performance with lower validation performance signals overfitting. Check for data leakage, duplicated samples across splits, label noise, class imbalance, and insufficient coverage of edge cases.

2. Audit model complexity: inspect neural-network depth, parameter count, and activation/gradient behavior. For genetic programming, record tree depth and node count; unconstrained program trees can grow large and memorize training examples. Verify that the evolutionary fitness does not accidentally reward complexity alone.

Reduce overfitting

Neural network: use L2/weight decay, dropout, early stopping, data augmentation where label-preserving, and a smaller architecture. Collect more diverse labelled data when possible. Prune weights/neurons only after verifying that held-out performance remains acceptable.

Genetic programming: limit tree depth or node count, add a parsimony penalty to fitness, and validate candidate programs on unseen folds. Penalize brittle or excessively complex expressions while preserving task performance.

Lower computational cost safely

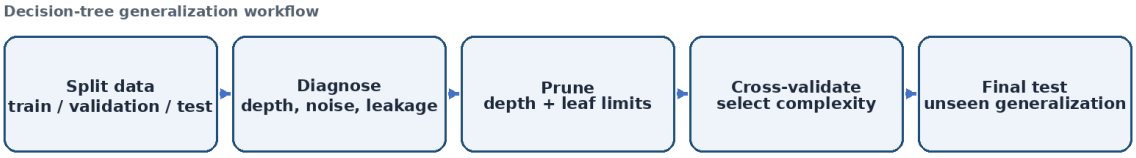
Cache repeated fitness evaluations, remove duplicate chromosomes/programs, evaluate candidates in parallel, and use staged or lower-cost proxy evaluation to discard clearly poor candidates before full training. Reuse model weights only where the comparison remains fair. Track wall-clock time, memory, inference latency, and validation metrics together. Any speed-up must be checked against a final untouched test set; reduced training cost is not useful if generalization degrades.

Conclusion: control capacity in both the neural network and evolved programs, and optimize the evaluation pipeline without allowing the test set to influence selection.

Use an experiment table to compare each change against a baseline: validation F1 or error, train-validation gap, training time, peak memory, and inference latency. For genetic programming, cache fitness by chromosome/tree hash so identical programs are not repeatedly evaluated, and use a complexity penalty to discourage expressions that add many nodes without meaningful improvement. For neural networks, mixed precision or smaller batches may reduce resource use, but confirm numerical stability and repeat the final evaluation with the same conditions for every competing model.

QUESTION 5

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)



Likely causes and diagnostic steps

Overfitting: a deep tree can create many small leaves that memorize noise or rare combinations. High training accuracy with low test accuracy is the classic signal of high variance, even when the dataset is large.

Data issues: check label quality, missing values, inconsistent categories, class imbalance, duplicates, and distribution shift. Search for leakage features that encode the target or use information that would not be available at prediction time. Use a split that matches deployment, such as time-based or group-based splitting when relevant.

Debugging sequence: compare training, validation, and test scores; inspect tree depth, number of leaves, minimum leaf size, and errors by class. Use stratified cross-validation when appropriate and keep the test set untouched until final model selection. Compare against a simple baseline to verify that the data pipeline is working.

Optimization methods

Pre-pruning: limit max_depth, max_leaf_nodes, or max_features, and increase min_samples_split/min_samples_leaf. These constraints prevent rules based on too few samples.

Post-pruning: cost-complexity pruning removes branches whose added complexity does not sufficiently improve predictive performance. Select the pruning strength using cross-validation, not the test set.

Feature selection and data quality: remove leakage and demonstrably irrelevant/redundant features using training folds only. Do not discard predictors solely because they seem weak; check their value with validation and domain knowledge. Handle imbalance with appropriate metrics or class weights if needed.

Final recommendation: select the simplest tree that performs consistently across validation folds, then report the held-out test metrics. The goal is not maximum training accuracy but stable performance on unseen cases.

Use precision, recall, F1, and a confusion matrix in addition to accuracy when classes are imbalanced; a model can appear accurate by mostly predicting the majority class. Inspect errors by subgroup to identify missing patterns or measurement issues. Tree impurity-based importance can favor high-cardinality features, so confirm important predictors with permutation importance on validation data or domain review. If performance varies strongly across folds, collect more representative data or report that uncertainty rather than choosing a single favorable split.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**30**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**10**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand